In [8]:
```
# import libraries
```

Background Radiation

In [187…
```
#Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

#1 Import data
data = pd.read_excel('Radioactive_Decay_template.xlsx',
                                sheet_name = 'Background')
Bdata = data['Latest: Radiation (Counts)'].to_numpy()
#Display data
display(Bdata)
# Exporting table...
data.to_latex("Bdata_table.tex", index=False)
```

```
array([2, 0, 5, 0, 6, 3, 4, 3, 1, 3, 1, 6, 2, 1, 3, 2, 2, 4, 1, 3, 3, 5,
       0, 3, 0, 1, 2, 1, 5, 3, 7, 4, 3, 4, 1, 5, 2, 3, 1, 0, 1, 4, 4, 3,
       3, 1, 3, 2, 3, 4, 3, 1, 2, 2, 2, 3, 4, 1, 2, 4, 7, 1, 4, 2, 2, 3,
       2, 0, 0, 3, 3, 2, 2, 5, 3, 1, 2, 5, 5, 3, 3, 2, 2, 4, 3, 4, 3, 5,
       3, 2, 3, 0, 2, 3, 3, 2, 3, 2, 3, 4])
```

In [176…
```
#2 Convert data from list to array(Not needed...)
#data = np.array(Bdata)
N = len(Bdata)

#3 Compute mean and sample standard deviation of the background counts
bkgAvg = np.mean(Bdata)
bkgStd = np.std(Bdata,ddof=1)

# Uncertainty in the mean
bkgErr = bkgStd / np.sqrt(N)

print(N)
print (bkgAvg)
print (bkgStd)
print (bkgErr)
type(data)
```

```
100
2.68
1.5431831198578285
0.15431831198578286
```

Out[176… pandas.core.frame.DataFrame

Create an histogram of the background [also called a frequency plot]

The data you collected represents a vector containing the number of counts detected (n) in each time interval. The best way to visualize this kind of data is in a histogram - a plot of the number of times (f_k) that we measured $n = k$ for each possible integer k.

As an example, assume you made 10 measurements and recorded the following counts: *n = [1 1 0 2 2 3 1 1 2 2]*. In this simple case, we have data that spans the range of integers k = [0,3] (inclusive). So our k vector will be *k = [0 1 2 3]* and our frequencies would be *f_k = [1 4 4 1]*.

Here are some functions that you may find useful:

Syntax: *k = numpy.arange(start, stop, step)*

This function creates a vector of evenly spaced numbers in the range *[start, stop)*, with step size *step*. By default, *step = 1*. Notice that the *stop* limit is excluded from the range, so to create the vector [0, 1, 2, 3], for example, you would use: *k = numpy.arange(0,4)*

The function

Syntax: *counts, bin_edges = numpy.histogram(n, k)*

returns two output arrays from NumPy:

• counts: An array of the number of data points that fall into each bin.

• bin_edges: A float array of length len(counts) + 1 containing the lower and upper boundaries of the bins.

Reference: *https://numpy.org/devdocs/reference/generated/numpy.histogram.html*

The matplotlib *hist* function both plots the data in a histogram, and computes the frequencies associated with each bin:

Syntax: *f_k, _ , _ = ax.hist(n, bins, color='skyblue', edgecolor = 'black')*

The first output *f_k* is the values of the histogram bins. Here, *n* is the input data, and *bins* defines the number and size of histogram bins to generate. If *bins* is a vector, it defines the edges of each histogram bin *including both the leftmost and rightmost edges*. This creates a subtle difference between the *bins* vector and the *k* vector. If you want your histogram bins to be centered on the integer values in *k*, *bins* will need to be offset by -0.5 and one element longer. In other words, if *k = [0 1 2 3]*, you'd want the *bins* vector to be *bins = [-0.5 0.5 1.5 2.5 3.5]*.

Reference: *https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.hist.html*

In [158…
```
#4 Define a figure to plot your data and line of best fit later in the file
fig1, ax1 = plt.subplots()

type(Bdata)
```

```
# Create an array of bins centered on integer values that span the entire ra
# Create an array of bins centered on integer values

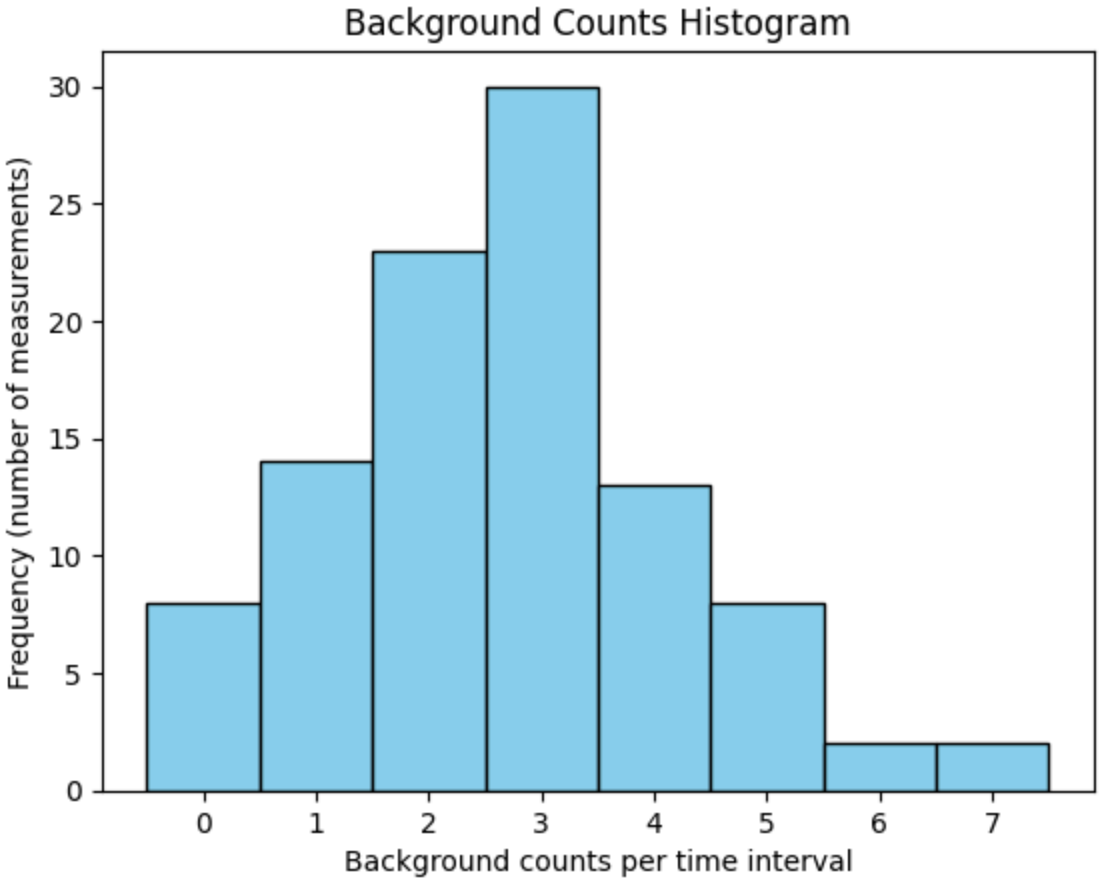
k = np.arange(Bdata.min(),
              Bdata.max() + 1)
bins =  np.arange(Bdata.min() - 0.5,
                  Bdata.max() + 1.5,
                  1)

#5 Create a histogram and plot it
f_k, _, _ = ax1.hist(Bdata, bins=bins, color="skyblue", edgecolor="black")

ax1.set_xlabel("Background counts per time interval")
ax1.set_ylabel("Frequency (number of measurements)")
ax1.set_title("Background Counts Histogram")

fig1.savefig("background_histogram.png", bbox_inches="tight")

plt.show()
```



Compare your histogram to a Poisson distribution

To compare your measurements to a Poisson distribution, overlay the frequency that you would expect to measure from a poisson distribution with the same average counts. The expected frequency is $f_k = Np_k$, where N is the total number of counts, and p_k is the Poisson probability distribution function with mean α given by:

$$p_k = \frac{\alpha^k}{k!} e^{-\alpha}$$

Note that the python package scypi.special has a built-in function to compute factorials:

Syntax: *from scipy.special import factorial*

A poisson distribution has the characteristic that $\sigma^2 = \alpha$ (the variance equals the mean). Is this true for your dataset?

In [175…

```
#6 Plot the expected bin frequencies on the same axes as your histogram
from scipy.special import factorial

bkgAvg = np.mean(Bdata)
N = len(Bdata)

# Calculate expected bin frequencies
expected_frequency = N * bkgAvg**k * np.exp(-bkgAvg) / factorial(k)

# Plot on the same axes as your histogram
ax1.plot(k, expected_frequency, "ro-", label="Poisson prediction")

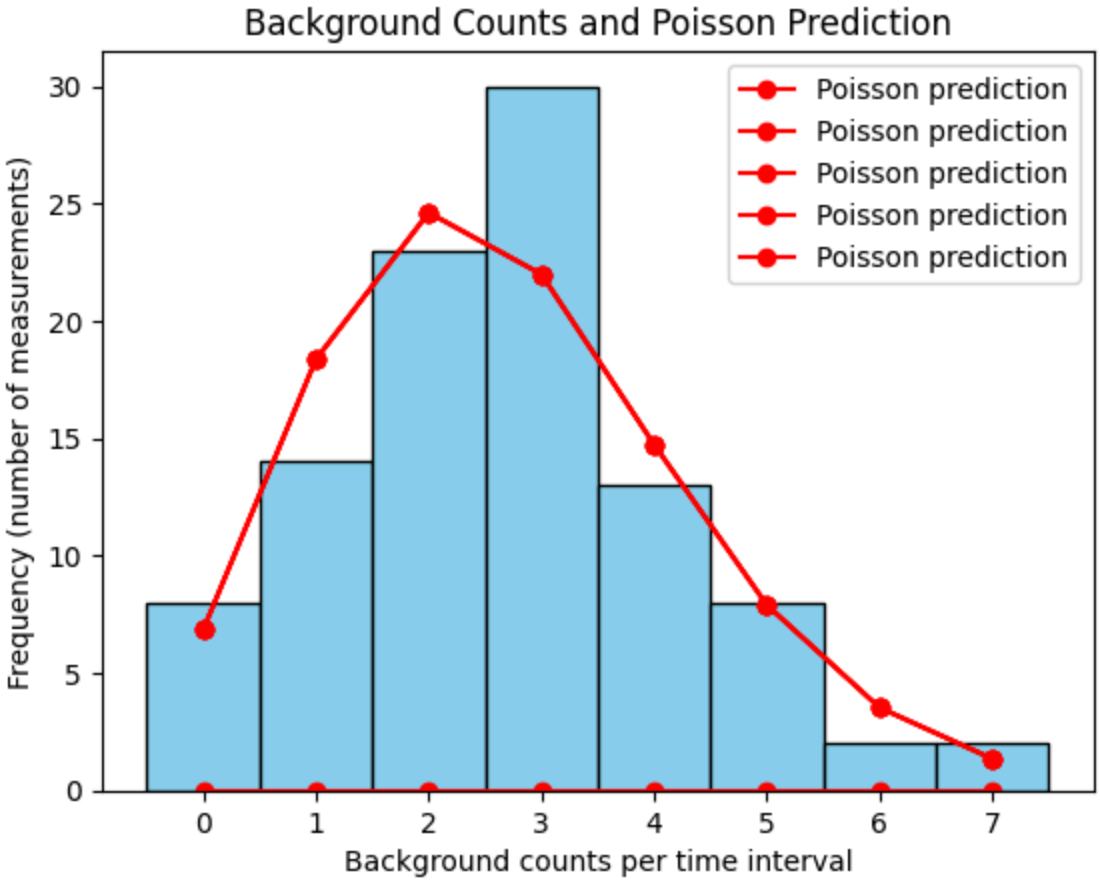
# Label axes and add a title
ax1.set_xlabel("Background counts per time interval")
ax1.set_ylabel("Frequency (number of measurements)")
ax1.set_title("Background Counts and Poisson Prediction")
ax1.legend()

fig1.savefig("BG_histogram with Fit.png", bbox_inches="tight")

display(fig1)

# Mean, Sample Variance, Standard Deviation, and Uncertainty in the Mean
bkgAvg = np.mean(Bdata)
bkgVar = np.var(Bdata, ddof=1)
bkgStd = np.std(Bdata)
# Uncertainty in the mean
bkgErr = bkgStd / np.sqrt(N)

print("Mean:", bkgAvg)
print("Sample Variance:", bkgVar)
print("Standard Deviation:", bkgStd)
print("Uncertainty in the Mean:", bkgStd)
```




```
Mean: 2.68
Sample variance: 2.381414141414141
Standard Deviation: 1.5354478174135386
```

The half life of ^{137m}Ba

In [188…

```
# Continue analyzing the rest of your Radioactive decay data
# 1. Extract half-life time and raw counts

# Load the HALF-LIFE sheet
data = pd.read_excel(
    'Radioactive_Decay_template.xlsx',
    sheet_name='Half-Life Ba137'
)
#Exporting table...
data.to_latex("HL_table.tex", index=False)

time = data['Time (s)'].to_numpy()
HLdata = data['Latest: Radiation (Counts)'].to_numpy()

# 2. Detector settings
tau = 90e-6   # Dead time: 90 microseconds, in seconds
dt = 5       # Each measurement lasts 5 seconds

# 3. Correct the half-life counts
HL_corrected = HLdata / (1 - (HLdata / dt) * tau)
```

```
# 4. Correct the BACKGROUND counts
# Bdata must contain your raw Part A background measurements!
background_corrected = Bdata / (1 - (Bdata / dt) * tau)

# Average corrected background counts per 5 seconds
bkgAvg_corrected = np.mean(background_corrected)

# 5. Subtract the corrected average background
corrected_counts = HL_corrected - bkgAvg_corrected

# 6. Plot
fig, ax = plt.subplots()

ax.plot(time, corrected_counts, 'o', markersize=3, label = "Data")

ax.set_xlabel('Time (s)')
ax.set_ylabel('Background-subtracted counts per 5 s')
ax.set_title('Barium-137m Decay — Dead-Time Corrected')

#plt.show()

# using curve_fit to fit the data
from scipy.optimize import curve_fit
def halfLife(t,A,L):
    return A*np.e**(-L*t)

popt, pcov = curve_fit(halfLife, time, corrected_counts, [250, 0.003466])
#print(popt)
A = popt[0]
L = popt[1]
print(A)
print(L)
def HLfit(t):
    return A*np.e**(-L*t)

t = np.linspace(0, 1200, 1200)
y = HLfit(t)

ax.plot(t, y, label='Exponential Fit')
ax.legend()
fig.savefig("Half-Life of Ba.png", dpi=300, bbox_inches="tight")

plt.show()

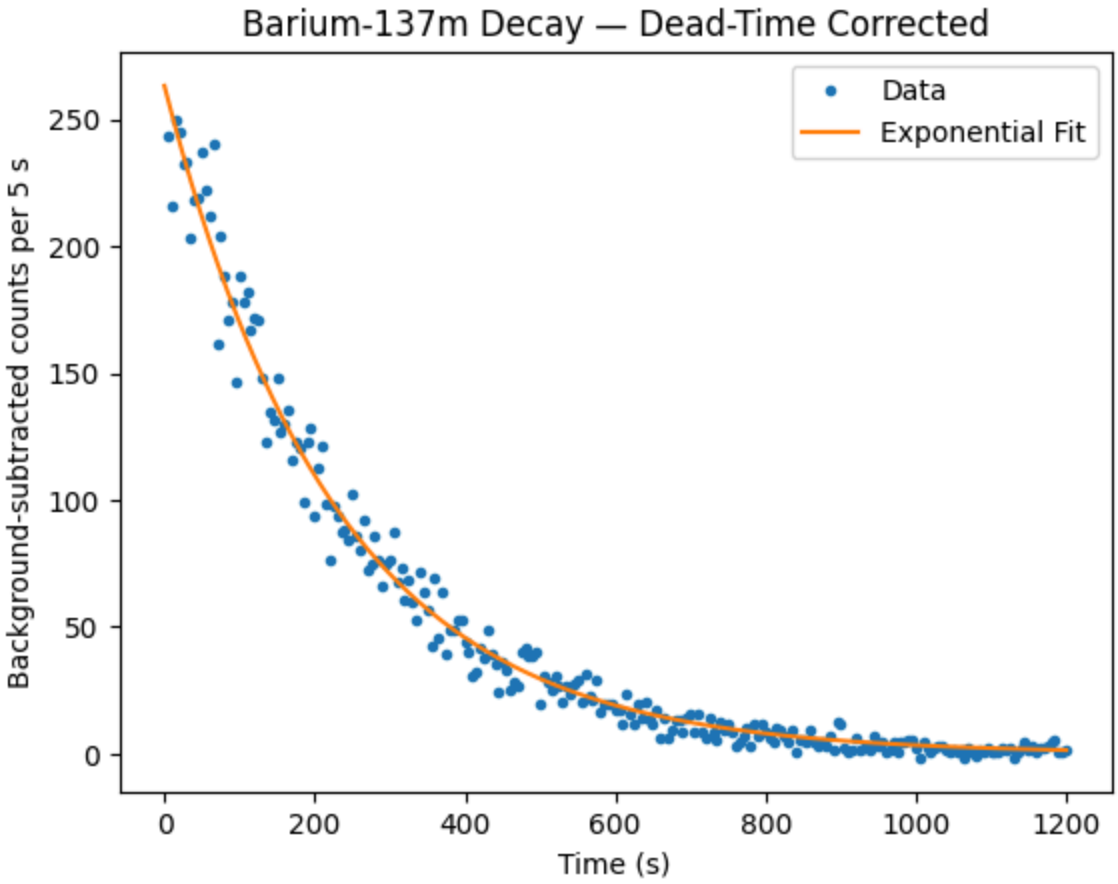
#Error on parameters...
HLpcov = np.sqrt(np.diag(pcov))

# Mean, Sample Variance, Standard Deviation, and Uncertainty in the Mean
HLAvg = np.mean(HLdata)
HLVar = np.var(HLdata, ddof=1)
HLStd = np.std(HLdata)
# Uncertainty in the mean
NHL = len(HLdata)
print(NHL)
HLErr = HLStd / np.sqrt(NHL)
```

```
print("Mean:", HLAvg)
print("Sample Variance:", HLVar)
print("Standard Deviation:", HLStd)
print("Uncertainty in the Mean:", HLErr)
print("Error on each Parameters:", HLpcov)


#Lambda is 0.0043773832515909585 so Half Life is approximately
#HL = ln2/0.0043773832515909585 is approximately 158.35 seconds
```

```
263.2813650787458
0.0043773832515909585
```




```
240
Mean: 51.50416666666667
Sample Variance: 4111.999982566248
Standard Deviation: 63.9911450851253
Uncertainty in the Mean: 4.1306106536572145
Error on each Parameters: [2.47516096e+00 5.76137806e-05]
```

Pb Absorption coefficient

In [189…

```
#Loading...
AC = pd.read_excel(
    'Radioactive_Decay_template.xlsx',
    sheet_name='Pb Absorption Coeff'
)

# Exporting table...
```

```
AC.to_latex("AC_table.tex", index=False)

display(AC)
thickness = AC['Thickness (cm)'].to_numpy()
Counts = AC['Counts'].to_numpy() - bkgAvg_corrected
#BGCorrected = Counts - Bdata
#print(thickness)
#print(Counts)
#plt.yticks(np.arange(0, 401, 10))

fig, ax = plt.subplots()

ax.plot(thickness, Counts, 'o', markersize=10, label='Data')


ax.set_xlabel('Time (s)')
ax.set_ylabel('Counts')
ax.set_title('Lead Absorbtion - Dead-Time Corrected')

#plt.plot(thickness, Counts, 'o', markersize=10)
#plt.show()

#fitting...
#I_naught...
print
I_o = 397 - bkgAvg_corrected
def I(x, u):
    return I_o*np.exp(-u*x)

popt, pcov = curve_fit(I, thickness, Counts, [1])

print(pcov)
U = popt[0]
print("The Absorbtion Coefficent is:", U)
def Ifit(t):
    return I_o*np.exp(-U*t)

t = np.linspace(0, 1200, 1200)
y = Ifit(t)

ax.plot(t, y, label='Exponential Fit')
fig.savefig("Lead Absorbtion with Fit.png", dpi=300, bbox_inches="tight")

plt.show()

#Errors...

#Error on parameters...
Pbpcov = np.sqrt(np.diag(pcov))

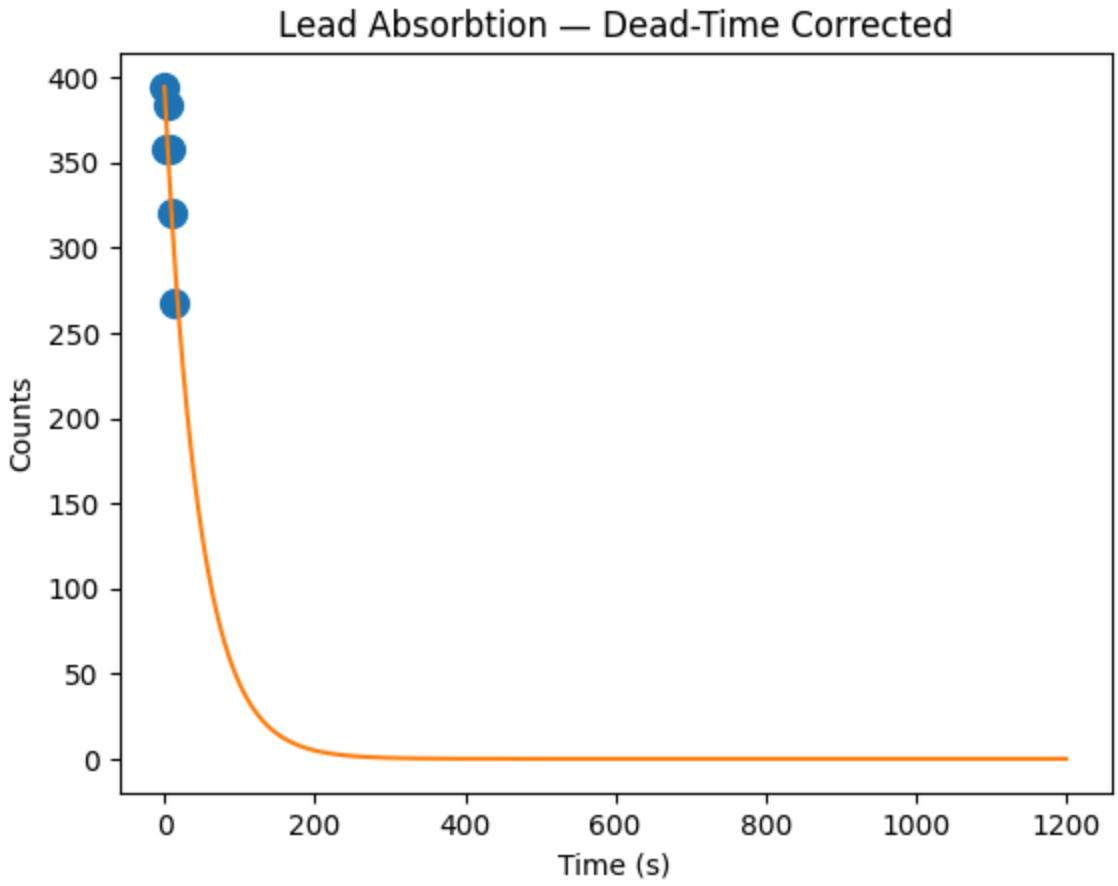
# Mean, Sample Variance, Standard Deviation, and Uncertainty in the Mean
#PbAvg = np.mean(HLdata)
#PbVar = np.var(HLdata, ddof=1)
#PbStd = np.std(HLdata)
# Uncertainty in the mean
#NHL = len(HLdata)
```

```
#print(NHL)
#HLErr = HLStd / np.sqrt(NHL)

#print("Mean:", HLAvg)
#print("Sample Variance:", HLVar)
#print("Standard Deviation:", HLStd)
#print("Uncertainty in the Mean:", HLErr)
print("Error on each Parameters:", Pbpcov)
```

	Thickness (cm)	Counts	Average Sheet Thickness (mm)	Trial 1	Trial 2	Trial 3	Minus Ziplock Bag
0	0.000000	397	0.000000	NaN	NaN	NaN	NaN
1	1.770000	361	1.770000	1.92	1.89	1.98	0.16
2	5.870000	386	4.100000	3.77	4.20	4.81	0.16
3	7.523333	360	1.653333	1.84	1.81	1.82	0.17
4	10.136667	323	2.613333	2.78	2.74	2.77	0.15
5	11.813333	270	1.676667	1.84	1.84	1.83	0.16

```
[[2.17131384e-05]]
The Absorbtion Coefficent is: 0.021932414677133864
```




```
Error on each Parameters: [0.00465974]
```

Inverse-square law

In [190…

```
#Load Data...
ISL = pd.read_excel(
    'Radioactive_Decay_template.xlsx',
    sheet_name='Inverse Square Law'
)
display(ISL)
# Exporting table...
ISL.to_latex("ISL_table.tex", index=False)

Dist = ISL['Distance (cm)'].to_numpy()
#the Background Average is subtracted...
Counts = ISL['Counts'].to_numpy() - bkgAvg_corrected*40 #per 5s times 40 = 2

#the geiger counter's dead time is ommitted because it is neglible

fig, ax = plt.subplots()

ax.plot(Dist, Counts, 'o', markersize=10)


ax.set_xlabel('Distance (cm)')
ax.set_ylabel('Counts')
ax.set_title('Inverse Square Law – Background Corrected')

#fitting...
#ISL...
#guessing for a using rearanged equation...
#y=a/(x-b)**n + c => a = y*x**2
a_guess = Counts[0] * Dist[0]**2
#print(a_guess)

#def ISL(x,a,b,c,n):
#    return a/(x-b)**n + c

#popt, pcov = curve_fit(ISL, Dist, Counts, [a_guess,0,0,2], maxfev=20000)
#print(Dist)
#print(Counts)

### Given equation not working!!! Manually setting b and c to zero...

def ISL2(x,a,n):
    return a/(x)**n

popt, pcov = curve_fit(ISL2, Dist, Counts, [a_guess,2], maxfev=20000)

#print(popt)
A = popt[0]
N = popt[1]

print("A is ",A)
print("N is ",N)

def ISL3(x):
    return A/(x)**N
```

```
t = np.linspace(Dist.min(), Dist.max(), 200)
y = ISL3(t)

ax.plot(t, y)
fig.savefig("Inverse Square Law with Fit.png", dpi=300, bbox_inches="tight")
plt.show()

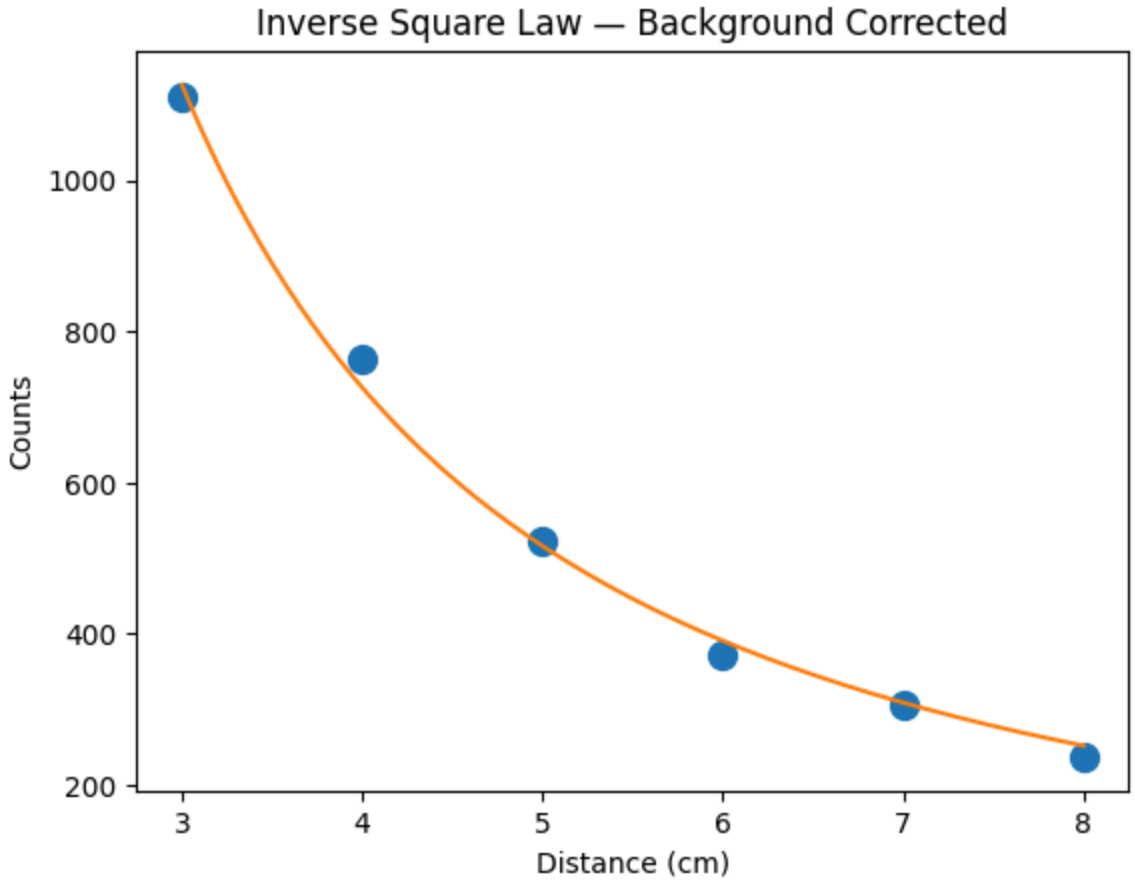
print("Finally, n is calculated to be:", N)

#Errors...

ISLpcov = np.sqrt(np.diag(pcov))
print("Error on each Parameters:", ISLpcov)
```

	Distance (cm)	Counts
0	3	1217
1	4	872
2	5	629
3	6	479
4	7	412
5	8	344

```
A is  6043.437008218625
N is  1.5288540467446783
```



```
Finally, n is calculated to be: 1.5288540467446783
Error on each Parameters: [4.56853766e+02 5.57902192e-02]
```

In [ ]: